

Plugin Architectures with Rust

an Analysis of Obstacles
and Prospects

20.05.2026



THU
Technische
Hochschule
Ulm

Contents



1	Introduction	4
1.a	Plugin Systems	5
1.b	Criteria	6
2	ABI	7
2.a	Basics	8
2.a.a	Application Binary Interface	8
2.a.b	Dynamic Linking/Loading	9
2.b	(Unstable) Rust ABI	10
2.b.a	Demonstration of dynamic linking and loading	10
2.b.b	 but it's Unstable	13
2.c	abi_stable crate	14
2.d	stabby crate	15
2.e	IPC	16
3	Conclusion	18
3.a	Performance	19
4	Appendix	20

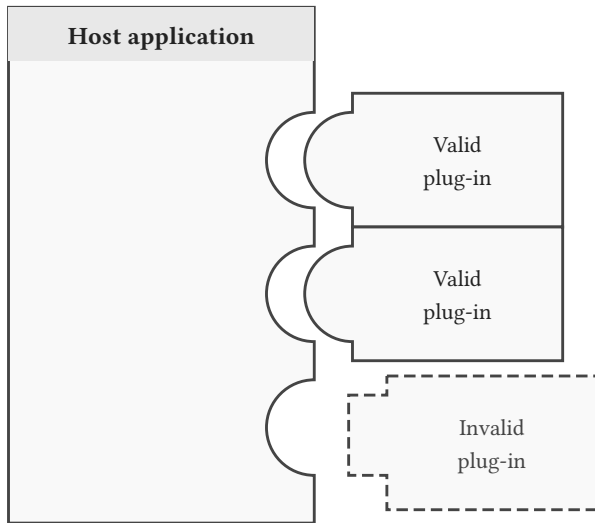
4.a	abi_stable implementation	21
4.a.a	Base Library	21
4.a.b	Plugin	23
4.a.c	Application	24
4.b	stabby implementation	25
4.b.a	Base Library	25
4.b.b	Plugin	26
4.b.c	Application	27

1 Introduction

Plugin Systems



- **Extensibility:** The ability to add new functionality post-deployment.
- **Modularity:** Plugins are isolated units that interact with the host through a well-defined interface.
- **Dynamic Loading:** Plugins are typically loaded at runtime rather than linked statically at compile time.



Criteria

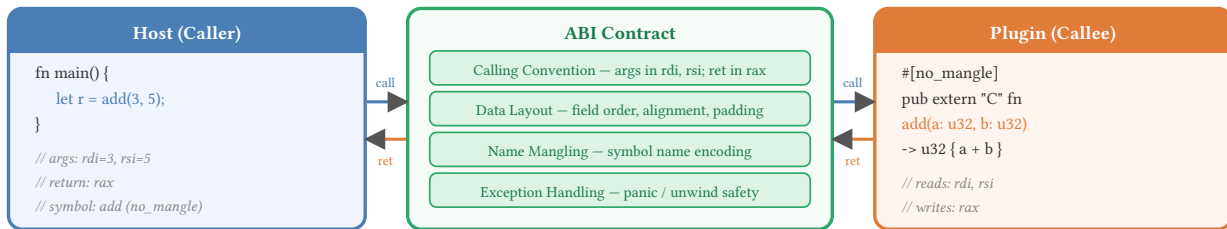


- Performance Overhead
- Development Complexity
- Language Limitations
- Interoperability

2 ABI

2.a.a Application Binary Interface

What an ABI Governs



Data Layout: struct { u8, u32, u8 }

■ field byte ■ padding

#[repr(C)] — 12 bytes



#[repr(Rust)] — may be 8 bytes



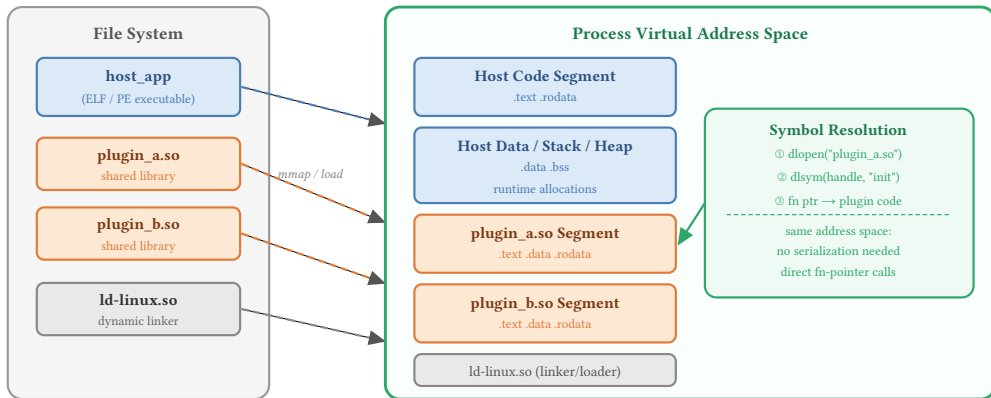
⊠ The compiler may reorder fields — a C caller reading offset 8 would get garbage

Basics (ii)



2.a.b Dynamic Linking/Loading

- 1) Discovery
- 2) Loading
- 3) Resolution
- 4) Execution



2.b.a Demonstration of dynamic linking and loading

```
// In base library (used by application and plugin):  
trait Fuser {  
    fn fuse(&self, data: &[SensorData]) -> Result<SensorData, Box<dyn Error>>;  
}
```

```
// In plugin (library):  
struct AveragePlugin;  
impl Fuser for AveragePlugin { ... }  
  
#[unsafe(no_mangle)] // ensure name can be found in binary  
pub fn average_plugin() -> Box<dyn Fuser> {  
    Box::new(AveragePlugin::new())  
}
```

(Unstable) Rust ABI (ii)



```
// In application:
type FuseFunc = extern "Rust" fn() -> Box<dyn Fuser>; // use Rust ABI
let plugin: Plugin<FuseFunc> = Plugin::new(
    &plugin_path.into(),
    "average_plugin".as_bytes()
).unwrap(); // 1), 2) and 3)

let fuser: Box<dyn Fuser> = plugin.get();
let sd = [ SensorData::new(24.0, 0.01), ... ];
let res = fuser.fuse(&sd); // 4) Execution
println!("{:?}", res);
```

(Unstable) Rust ABI (iii)



```
// In application:
struct Plugin<T> where T: Fn() -> Box<dyn Fuser> + Copy{
    lib: Library,
    func: T, // or Vec<T> or HashMap<String, T>
}
```

Using the "libloading" crate:

- 1) Discovery
- 2) Loading
- 3) Resolution
- 4) Execution

```
fn new(path: &PathBuf, s: &[u8]) -> Result<Self, libloading::Error> {
    let lib = unsafe { Library::new(path) }?; // 1+2
    let symbol: Symbol<T> = unsafe { lib.get(s) }?; // 3)
    let func: T = *symbol; // fn deref(&self) -> &T
    Ok(Plugin { lib, func })
}
fn get(&self) -> Box<dyn Fuser> { (self.func)() }
```



2.b.b but it's Unstable

- Compiler version changes **may** impact the generated binary
- Base library, application and plugin need to run on the **same compiler version**

- Widely used crate with a lot of community support
-

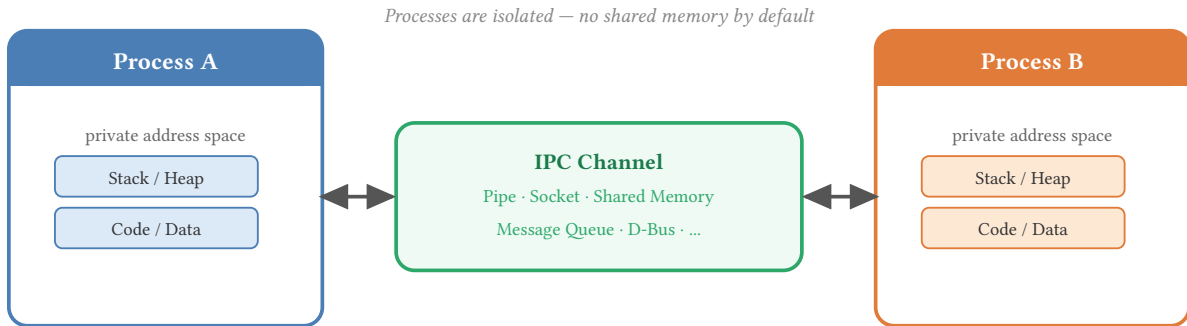
but:

- Many hidden structs and types are generated, and therefore overwhelming at first

- A lot simpler to get started
- Good explanation of the concepts behind ABI development (and why Rust doesn't like the C ABI)

but:

- relatively new and therefore not a lot of community engagement (yet)
- had some breaking changes due to a rust compiler update



Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua quaerat.

Term
Definition

3 Conclusion

Crate	Perf. Diff	Development Complexity	Limitation	Interoperability
native		++	0	0
unstable_abi	16%	--	--	0
abi_stable	25%	0	0	+
stabby	9%	0	0	+
rust_bridge (JSON)	537%	+	0	++
rust_bridge (Binary)	25%	-	0	++

Table 1: Summary of results.

4 Appendix

4.a.a Base Library

```
#[sabi_trait] // converts this to: Fuser_T0<'lt, Pointer<()>> (and more...)
pub trait Fuser {
    #[sabi(last_prefix_field)]
    fn fuse<'a>(&self, data: RSlice<'a, SensorData>) -> RResult<SensorData, RBoxError>;
}
pub type FuserBox = Fuser_T0<'static, RBox<()>>;
```

```
#[repr(C)]
#[derive(StableAbi)]
#[sabi(kind(Prefix(prefix_ref = FuserLibRef)))]
#[sabi(missing_field(panic))]
pub struct FuserLib{
    pub new_fuser: extern "C" fn() -> FuserBox,
}
```

abi_stable implementation (ii)



```
impl RootModule for FuserLibRef {  
    abi_stable::declare_root_module_statics! {FuserLibRef}  
    const BASE_NAME: &'static str = "fuser_lib";  
    const NAME: &'static str = "fuser_lib";  
    const VERSION_STRINGS: VersionStrings = package_version_strings!();  
}
```

```
pub fn load_root_module_in_directory(directory: &Path) -> Result<FuserLibRef,  
LibraryError> {  
    FuserLibRef::load_from_directory(directory)  
}
```



4.a.b Plugin

```
// in plugin:
#[export_root_module]
fn average_plugin() -> FuserLibRef {
    FuserLib {new_fuser: average_plugin_impl}.leak_into_prefix()
}
#[sabi_extern_fn]
fn average_plugin_impl() -> FuserBox {
    Fuser_T0::from_value(AveragePlugin::new(),
                        TD_Opaque, // disallow DOWNCASTING of trait objects
    )
}
```



4.a.c Application

```
let path : PathBuf = plugin_path.into();
let data = [ SensorData::new(24.0, 0.01), ... ];
let data = RSlice::from_slice(&data);

let plugin: FuserLibRef = load_root_module_in_directory(&path)?;
let fuser: Fuser_T0<'_, RBox<()>> = plugin.new_fuser>();
let res= plugin.fuse(data);
println!("a) {:?}", res);
```




4.b.a Base Library

```
#[stabby::stabby(chcked)] // neccessary for stabby!
pub trait Fuser {
    extern "C" fn fuse(
        &self,
        data: stabby::slice::Slice<'_, SensorData>,
    ) -> stabby::result::Result<SensorData, error::Error>;
    extern "C" fn print(&self);
}
```

stabby implementation (ii)



4.b.b Plugin

```
#[stabby::stabby]
pub struct AveragePlugin { }
impl Fuser for AveragePlugin { ... }

#[stabby::export]
pub extern "C" fn average_plugin()
    -> stabby::dynptr!(Box<dyn Fuser>) {
    Box::new(AveragePlugin::new()).into()
}
```

stabby implementation (iii)



4.b.c Application

```
pub type FuseFunc = extern "C" fn() -> FuserPluginDyn;
pub type FuserPluginDyn = stabby::dynptr!(stabby::boxed::Box<dyn
app_core::fuse::Fuser>);

pub struct Plugin<TDyn> {
    // Keep the plugin value before the library handle so it is dropped first.
    func: TDyn,
    _lib: Library,
}

impl<TDyn> Plugin<TDyn> {
    pub fn new<TFunc>(
        plugin_name: &PathBuf,
        symbol: &[u8],
    ) -> Result<Self, Box<dyn Error + Send + Sync>>
```

stabby implementation (iv)



```
where
    TFunc: PluginFactory<Dyn = TDyn>, // allows to use of "get(&self)"
{
    let lib = unsafe { libloading::Library::new(plugin_name)? };
    let my_imported_function = unsafe { lib.get_stabbied::<TFunc>(symbol)? };
    let fund = *my_imported_function;
    let tdyn = fund.get();

    Ok(Self {
        func: tdyn,
        _lib: lib,
    })
}

pub fn get(&self) -> &TDyn {
    &self.func
}
```

stabby implementation (v)



```
pub fn get_mut(&mut self) -> &mut TDyn {
    &mut self.func
}

pub fn get_plugin<TFunc>(
    plugin_name: PathBuf,
    symbol: &[u8],
) -> Result<Plugin<TFunc::Dyn>, Box<dyn Error + Send + Sync>>
where
    TFunc: PluginFactory,
{
    Plugin:::<TFunc::Dyn>::new:::<TFunc>(&plugin_name, symbol)
}

let mut plugin: Plugin<FuserPluginDyn> = get_plugin:::<FuseFunc>(plugin_path.into(),
```

stabby implementation (vi)



```
"average_plugin".as_bytes()).unwrap();  
let data = [ SensorData::new(24.0, 0.01), ... ];  
let data: Slice<'_, SensorData> = Slice::new(&data);  
let fuser = plugin.get();  
let res = fuser.fuse(data);  
println!("a {:?}", res);
```

